

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 07 -

PADRONIZAÇÃO DE CÓDIGO DE MODELOS DE ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	17

EMSE

O QUE VEM POR AÍ?

Projetos de Machine Learning são moldados por decisões técnicas que vão além da escolha de algoritmos e métricas de desempenho. À medida que soluções evoluem, a forma como o código é estruturado, documentado e mantido passa a exercer papel central na confiabilidade e na longevidade dos resultados obtidos. A ausência de padrões dificulta a compreensão dos experimentos, compromete a reprodutibilidade e limita a colaboração entre diferentes atores envolvidos no projeto.

Por outro lado, práticas consistentes de organização, nomenclatura, documentação e versionamento tornam explícitas as escolhas metodológicas e permitem que modelos sejam compreendidos, avaliados e evoluídos de maneira sistemática. Nesse contexto, a padronização emerge como elemento fundamental para transformar experimentos isolados em projetos tecnicamente maduros, sustentáveis e alinhados às exigências acadêmicas e profissionais.

HANDS ON

A padronização de código em projetos de Machine Learning constitui um elemento central para a construção de soluções técnicas confiáveis, compreensíveis e sustentáveis. À medida que projetos evoluem, decisões relacionadas à escrita, organização e documentação do código passam a influenciar diretamente a reprodutibilidade dos experimentos. Essas decisões também impactam a colaboração entre diferentes profissionais e a manutenção das soluções ao longo do tempo. A ausência de padrões tende a introduzir ambiguidades, dificultar a interpretação dos resultados e comprometer a evolução dos projetos. Nesse contexto, a padronização surge como um mecanismo de organização metodológica e técnica.

O estilo de codificação é tratado como instrumento de comunicação, no qual variáveis, funções e classes representam conceitos do domínio do problema. A clareza na nomenclatura e a consistência estrutural favorecem a leitura e reduzem erros conceituais em pipelines de Machine Learning. A adoção consciente de guias de estilo contribui para uniformidade e previsibilidade do código. A organização de projetos permite separar responsabilidades e tornar explícitos os fluxos de dados, modelos e avaliações. Estruturas bem definidas facilitam a manutenção, a reutilização e a extensão das soluções desenvolvidas.

A documentação assume papel fundamental ao registrar decisões técnicas, suposições e limitações de forma objetiva. O uso criterioso de docstrings e comentários contribui para a compreensão do raciocínio por trás das implementações. O controle de versão amplia-se para além do código, incorporando dados, modelos e configurações experimentais. Essa abordagem assegura rastreabilidade e reforça princípios de ciência reproduzível. A gestão estruturada de experimentos permite comparar resultados e compreender a evolução dos modelos e a colaboração em projetos de Machine Learning é fortalecida por meio da revisão de código. Esse processo atua como mecanismo de validação técnica e aprendizado coletivo. A identificação de problemas conceituais torna-se mais eficiente em ambientes colaborativos, a definição de padrões internos reflete maturidade técnica e alinhamento metodológico e a análise crítica de código consolida uma postura profissional orientada à qualidade. Esses elementos contribuem para o desenvolvimento responsável de soluções em Machine Learning.

SAIBA MAIS

A importância da padronização em projetos de Machine Learning

Projetos de Machine Learning envolvem mais do que a simples construção de modelos capazes de gerar boas previsões. Eles fazem parte de sistemas maiores, são continuamente modificados, avaliados e reaproveitados e geralmente envolvem múltiplos(as) profissionais ao longo do tempo. Nesse contexto, a forma como o código é escrito, organizado e documentado torna-se tão relevante quanto os resultados obtidos.

A padronização de código consiste na definição de convenções e boas práticas que orientam a escrita, a organização e a manutenção do código-fonte. Em Machine Learning, essas convenções assumem um papel central, pois lidam não apenas com lógica de programação, mas também com dados, experimentos, modelos e métricas, que precisam ser compreendidos e reproduzidos posteriormente.

A adoção de padrões não tem como objetivo restringir a flexibilidade do desenvolvimento, mas sim estabelecer uma base comum que favoreça clareza, consistência e previsibilidade em projetos de médio e longo prazo.

Código funcional e código sustentável

Um equívoco recorrente em projetos de dados é considerar que um código está adequado apenas porque produz o resultado esperado. Embora a corretude seja essencial, ela não é suficiente para garantir qualidade em um contexto profissional ou acadêmico. Código sustentável em Machine Learning apresenta características adicionais, como:

- Facilidade de leitura e compreensão.
- Organização lógica clara.
- Separação de responsabilidades.
- Capacidade de manutenção e evolução.
- Possibilidade de reprodução dos resultados.

Um código que atende apenas ao critério de funcionamento tende a se tornar frágil à medida que o projeto cresce, dificultando ajustes, validações e colaborações futuras.

Considere um projeto em que todo o processo de leitura de dados, tratamento, treinamento e avaliação do modelo esteja concentrado em um único script ou notebook extenso, com nomes genéricos de variáveis e ausência de documentação. Embora o modelo possa ser treinado corretamente, a compreensão do fluxo completo torna-se complexa para terceiros — e até para o próprio autor após algum tempo.

Em contraste, um projeto estruturado em módulos, com funções bem definidas, nomes claros e documentação mínima, permite que o mesmo experimento seja compreendido, reproduzido e modificado com menor esforço.

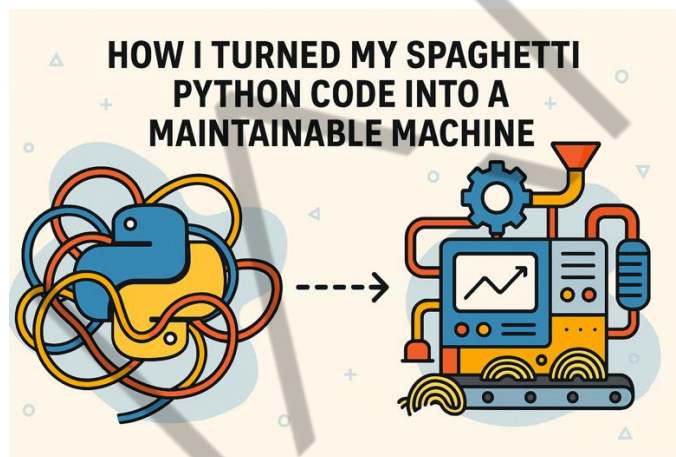


Figura 1 – Código funcional e código sustentável
Fonte: Rajput (2025)

Consequências da ausência de padrões

A falta de padronização em projetos de Machine Learning gera impactos diretos na qualidade e na longevidade do código, como dificuldades de manutenção. Sem padrões bem definidos:

- Funções acumulam múltiplas responsabilidades.
- Ajustes simples exigem alterações em vários pontos do código.
- A identificação da origem de erros torna-se mais difícil.

Esses fatores elevam o custo de manutenção e aumentam a probabilidade de falhas.

A reprodutibilidade é um princípio fundamental em ciência de dados. Projetos sem padronização frequentemente apresentam:

- Dependências implícitas.
- Hiperparâmetros não documentados.
- Ambiguidade sobre versões de dados e modelos utilizados.

Como consequência, resultados obtidos em um momento específico podem não ser reproduzidos posteriormente, comprometendo a confiabilidade do trabalho.

Em ambientes colaborativos, a ausência de padrões dificulta a integração entre membros da equipe. Cada desenvolvedor(a) tende a adotar estilos e estruturas próprias, o que aumenta o tempo necessário para entendimento do projeto e torna revisões de código mais complexas e subjetivas.

A padronização atua como um mecanismo de alinhamento, reduzindo a dependência de conhecimento tácito e favorecendo a comunicação técnica.

Padronização e ciência reproduzível

A ciência reproduzível exige que experimentos possam ser compreendidos e replicados por terceiros. Em Machine Learning, isso implica não apenas a disponibilização do código, mas também sua organização e clareza. Padrões bem definidos contribuem para:

- Transparência dos processos.
- Clareza na definição de experimentos.
- Registro explícito de decisões técnicas.
- Maior confiabilidade dos resultados obtidos.

Dessa forma, a padronização deixa de ser apenas uma prática de engenharia de software e passa a ser um requisito metodológico.

O estudo da padronização de código em projetos de Machine Learning visa desenvolver a capacidade de produzir soluções técnicas que sejam compreensíveis, sustentáveis e alinhadas às exigências acadêmicas e profissionais.

A consolidação de boas práticas permite que modelos e experimentos sejam tratados como ativos de longo prazo, passíveis de evolução, auditoria e reaproveitamento, contribuindo para a maturidade dos projetos e das equipes envolvidas.

Estilo de Codificação Unificado em Projetos de Machine Learning

Antes de discutir padrões e ferramentas, é importante compreender que o código, em projetos de Machine Learning, não serve apenas para executar instruções, mas também para comunicar ideias entre pessoas. Ele precisa ser entendido por outros membros da equipe, revisores técnicos, futuros mantenedores e, frequentemente, pelo próprio autor após algum tempo.

Um estilo de codificação consistente permite que o leitor compreenda rapidamente o propósito e o funcionamento do código, concentrando-se na lógica do problema e não na forma como ele foi escrito. Isso melhora a colaboração, reduz erros e torna o desenvolvimento mais eficiente.

Uma variável é um nome associado a um valor armazenado em memória. Em Machine Learning, variáveis podem representar dados brutos, dados processados, conjuntos de treino e teste, parâmetros, modelos e métricas. Por isso, seus nomes devem refletir claramente o significado do conteúdo que armazenam.

Nomes mal escolhidos dificultam a leitura e aumentam o risco de erros conceituais, enquanto nomes descritivos tornam o pipeline mais compreensível e autoexplicativo. Convenções como `raw_data`, `processed_data`, `encoded_features`, `scaled_features`, `X_train` e `y_test` ajudam a indicar o estágio e o papel dos dados dentro do processo. Essa padronização facilita a leitura, a manutenção e a evolução do código.

Funções são utilizadas para organizar o código em blocos reutilizáveis e compreensíveis. Uma função bem definida deve ter uma única responsabilidade e um nome que descreva claramente a ação que executa. Nomes genéricos dificultam o entendimento e a reutilização, enquanto nomes descritivos permitem compreender rapidamente o comportamento da função sem necessidade de comentários

adicionais. Essa prática melhora a legibilidade, facilita testes e contribui para um código mais modular e confiável.

Classes são utilizadas quando é necessário representar entidades mais complexas, como pipelines de dados, modelos e avaliadores. Elas permitem agrupar dados e comportamentos relacionados, promovendo melhor organização e encapsulamento. Em Python, classes seguem o padrão CamelCase e seus nomes devem refletir claramente sua responsabilidade. Essa forma de modelagem ajuda a estruturar o sistema de maneira mais clara, facilitando manutenção, reutilização e evolução do projeto.

O PEP 8 é o guia oficial de estilo da linguagem Python e define recomendações para nomeação, organização e formatação do código. Ele estabelece padrões que tornam o código mais legível, consistente e fácil de manter. O objetivo do PEP 8 não é impor regras arbitrárias, mas garantir que códigos escritos por diferentes desenvolvedores(as) sigam uma estrutura previsível. Seguir esse padrão melhora a clareza do código e facilita o trabalho colaborativo.

Em projetos reais, a padronização do código é reforçada pelo uso de ferramentas automatizadas que ajudam a formatar o código, detectar inconsistências e identificar problemas simples antes da execução. Essas ferramentas contribuem para manter a consistência do projeto mesmo em equipes grandes, reduzindo a dependência de revisão manual e melhorando a qualidade geral do código.

O estilo de codificação deve ser entendido como parte integrante da prática profissional em Machine Learning. Ele contribui diretamente para a legibilidade, a manutenção e a confiabilidade dos experimentos. Ao adotar padrões claros e consistentes, o código deixa de ser um artefato individual e passa a ser um componente estruturado e compreensível dentro de um sistema maior.

Organização de Projetos de Machine Learning

Projetos de Machine Learning tendem a crescer rapidamente em complexidade. O que começa como um experimento simples pode evoluir para um sistema com múltiplos arquivos, modelos e conjuntos de dados. Sem uma organização clara, torna-se difícil compreender, manter e reproduzir os resultados.

Uma estrutura bem definida permite identificar rapidamente onde os dados são processados, onde o modelo é treinado e onde os resultados são gerados, facilitando o desenvolvimento e a manutenção.

Um projeto de Machine Learning envolve diversos componentes, incluindo dados, pré-processamento, definição de modelos, treinamento, avaliação, configurações e resultados. Uma boa organização reflete esses componentes na estrutura do projeto, tornando o fluxo de trabalho explícito e compreensível.

Um problema frequente é o uso excessivo de notebooks como principal estrutura do projeto. Embora úteis para exploração, notebooks dificultam versionamento, reutilização e execução consistente. Outro problema comum é concentrar múltiplas responsabilidades em um único script, o que dificulta a manutenção e aumenta o risco de erros.

A organização de projetos deve seguir o princípio da separação de responsabilidades, no qual cada componente possui uma função clara. O código de leitura de dados deve ser separado do código de treinamento e o código de avaliação deve ser independente das transformações. Essa separação melhora a modularidade e facilita manutenção e testes.

Uma estrutura organizada geralmente inclui diretórios separados para dados, modelos, avaliação, utilidades, configurações e notebooks. Essa separação permite localizar facilmente cada componente e evita a mistura de responsabilidades, tornando o projeto mais compreensível e sustentável.

Projetos mal organizados apresentam arquivos com nomes genéricos, mistura de dados e código e falta de estrutura clara. Em contraste, projetos bem organizados possuem estrutura modular, facilitam a manutenção, melhoram a colaboração e tornam o fluxo de trabalho mais transparente.

Documentação e Comentários Padronizados em Projetos de Machine Learning

Em Machine Learning, o código representa não apenas operações computacionais, mas também decisões metodológicas. A documentação permite registrar o propósito do código, suas entradas, saídas e suposições, facilitando compreensão e manutenção.

A documentação pode ser estrutural, descrevendo módulos e classes; funcional, explicando o comportamento de funções; ou composta por comentários pontuais que esclarecem decisões específicas.

Docstrings são descrições incorporadas diretamente no código e permitem documentar funções e classes de forma padronizada. Elas facilitam o entendimento e permitem o uso de ferramentas automáticas de documentação. Existem diferentes estilos de docstring, como o estilo NumPy e o estilo Google. O mais importante é manter consistência ao longo do projeto.

Comentários devem explicar o motivo de uma decisão, não apenas repetir o que o código já expressa. Comentários bem utilizados ajudam a compreender escolhas técnicas importantes. Comentários como TODO e FIXME são utilizados para indicar melhorias futuras ou problemas conhecidos, contribuindo para a manutenção e a evolução do sistema. Já documentação adequada facilita a integração de novos membros, melhora a reprodutibilidade e garante maior confiabilidade ao longo do tempo.

Controle de Versão e Gestão de Experimentos em Projetos de Machine Learning

Controle de versão é o conjunto de práticas e ferramentas utilizadas para registrar, acompanhar e gerenciar mudanças realizadas em um projeto ao longo do tempo. Em termos simples, trata-se de manter um histórico confiável de o que foi alterado, quando, por quem e por quê.

Em projetos tradicionais de software, o controle de versão costuma se concentrar no código-fonte. Em Machine Learning, no entanto, essa abordagem precisa ser ampliada, pois os resultados dependem não apenas do código, mas também dos dados, dos modelos treinados e das configurações utilizadas em cada experimento.

Projetos de Machine Learning possuem características que tornam o controle de versão mais desafiador:

- O mesmo código pode produzir resultados diferentes com dados distintos.
- Pequenas alterações em hiperparâmetros podem gerar grandes impactos nas métricas.

- Modelos treinados são artefatos que precisam ser armazenados e comparados.
- Experimentos são frequentemente iterativos e exploratórios.

Por esse motivo, versionar apenas o código não é suficiente para garantir rastreabilidade e reprodutibilidade.

Além disso, o versionamento de código permite:

- Recuperar versões anteriores em caso de erro.
- Entender a evolução do projeto.
- Trabalhar de forma colaborativa sem sobrescrever o trabalho de outros.

Em Machine Learning, os dados utilizados no treinamento são tão importantes quanto o código. Alterações nos dados — mesmo que pequenas — podem mudar completamente o comportamento do modelo. Versionar dados permite:

- Reproduzir experimentos passados
- Comparar resultados entre diferentes versões de datasets
- Auditar decisões tomadas com base em determinados dados

Modelos treinados devem ser tratados como artefatos versionáveis, assim como o código. Cada modelo representa uma combinação específica de dados, algoritmo, hiperparâmetros e decisões de pré-processamento. Sem versionamento, torna-se difícil saber qual modelo gerou determinado resultado em produção ou em um relatório.

Já um experimento em Machine Learning corresponde a uma execução específica de um pipeline, definida por um conjunto de dados, uma configuração de modelo e um conjunto de métricas avaliadas. Gerenciar experimentos significa manter um registro estruturado dessas execuções.

O principal objetivo do controle de versão e da gestão de experimentos é garantir reprodutibilidade. Um experimento reprodutível é aquele que pode ser executado novamente, gerando resultados compatíveis, mesmo após semanas ou meses. A reprodutibilidade fortalece a confiabilidade científica, a transparência técnica e a credibilidade dos projetos de Machine Learning.

Em projetos maduros, o controle de versão deixa de ser uma atividade pontual e passa a integrar a cultura de desenvolvimento. Em Machine Learning, essa prática é fundamental para lidar com a complexidade inerente a dados, modelos e experimentos.

Ao adotar uma abordagem estruturada para versionamento e gestão de experimentos, projetos tornam-se mais organizados, auditáveis e sustentáveis ao longo do tempo.

Revisão de Código e Colaboração em Projetos de Machine Learning

Projetos de Machine Learning são desenvolvidos de forma colaborativa, exigindo práticas que garantam qualidade e consistência. Já revisão de código é o processo de analisar alterações antes de integrá-las ao sistema, garantindo qualidade e alinhamento com padrões. A revisão ajuda a identificar erros conceituais, vazamento de dados e avaliações incorretas, que podem comprometer resultados. Além disso, a revisão promove troca de conhecimento e desenvolvimento técnico coletivo. Erros conceituais, avaliações incorretas e falhas metodológicas podem ser identificados durante revisões. A revisão deve verificar clareza, organização, documentação e reprodutibilidade.

Padrões Internos e Guias de Estilo em Projetos de Machine Learning

À medida que projetos de Machine Learning amadurecem, torna-se inviável depender apenas de boas intenções individuais para manter qualidade e consistência. Mesmo quando todos dominam boas práticas, interpretações distintas podem levar a estilos divergentes de código, organização e documentação.

Padrões internos surgem para resolver esse problema. Eles funcionam como um acordo técnico explícito, definindo como modelos, dados e experimentos devem ser estruturados dentro de um contexto específico. Diferentemente de guias genéricos, padrões internos levam em conta o tipo de problema abordado, as tecnologias utilizadas, o nível de maturidade da equipe e os objetivos do projeto.

Guias de estilo amplamente adotados fornecem uma base sólida para a padronização, mas raramente atendem perfeitamente às necessidades de projetos de Machine Learning sem adaptações.

Esses guias devem ser vistos como referências técnicas, não como regras rígidas. O valor está em compreender seus princípios e adaptá-los de forma consciente ao contexto do projeto.

Um dos guias mais conhecidos define boas práticas para nomenclatura, organização de código, formatação, clareza e legibilidade. Essas recomendações ajudam a criar códigos previsíveis e fáceis de manter, especialmente em projetos colaborativos.

No entanto, esses guias são generalistas e não abordam aspectos específicos de Machine Learning, como pipelines de dados, versionamento de modelos, rastreabilidade de experimentos e organização de métricas.

Projetos de Machine Learning apresentam desafios que não são contemplados diretamente por guias de estilo tradicionais, como:

- Convenções para dados em diferentes estágios (brutos, processados, transformados).
- Padronização de nomes para experimentos.
- Estruturação de pipelines de treino e avaliação.
- Documentação de decisões experimentais.

Sem adaptações, equipes tendem a aplicar boas práticas de forma inconsistente ou incompleta.

Padrões internos devem responder a perguntas práticas, como:

- Como nomear variáveis de dados em diferentes etapas?
- Onde definir hiperparâmetros?
- Como salvar e identificar modelos treinados?
- Como organizar experimentos exploratórios?

Responder a essas questões de forma explícita reduz ambiguidades e acelera o desenvolvimento.

O QUE VOCÊ VIU NESTA AULA?

A padronização de código em projetos de Machine Learning representa um componente essencial para a construção de soluções técnicas confiáveis, compreensíveis e sustentáveis. Em contextos nos quais modelos evoluem continuamente, são reutilizados ou avaliados por diferentes pessoas, a forma como o código é escrito, organizado e documentado exerce influência direta sobre a reprodutibilidade dos experimentos, a colaboração entre profissionais e a manutenção das soluções ao longo do tempo.

O estilo de codificação assume papel central nesse processo ao funcionar como instrumento de comunicação técnica. Variáveis, funções e classes não representam apenas estruturas computacionais, mas conceitos associados a dados, modelos e processos de transformação. A clareza na nomenclatura reduz ambiguidades semânticas e facilita a compreensão de pipelines de Machine Learning, especialmente em projetos complexos.

A organização de projetos de Machine Learning é abordada como um elemento estruturante que reflete a própria lógica do processo de desenvolvimento. Projetos bem organizados tornam explícitas as etapas de coleta e preparação de dados, definição de modelos, treinamento, avaliação e persistência de resultados. A separação clara de responsabilidades reduz o acoplamento entre componentes, facilita a manutenção e possibilita a reutilização de partes do código em diferentes contextos.

A documentação desempenha papel fundamental ao registrar decisões técnicas, suposições adotadas e limitações conhecidas. Em projetos de Machine Learning, nos quais escolhas metodológicas impactam diretamente os resultados, documentar significa tornar explícito o raciocínio por trás das implementações. O uso criterioso de docstrings permite descrever entradas, saídas e comportamentos esperados de funções e classes, enquanto comentários pontuais esclarecem decisões que não são imediatamente evidentes pela leitura do código.

O controle de versão amplia-se, nesse contexto, para além do código-fonte. Em Machine Learning, resultados dependem não apenas da implementação, mas também dos dados utilizados, das configurações experimentais e dos modelos treinados.

Versionar esses elementos de forma estruturada asseguram rastreabilidade e permite compreender a evolução dos experimentos ao longo do tempo.

A definição de padrões internos surge como etapa natural em projetos mais maduros. Esses padrões representam a consolidação de boas práticas adaptadas ao contexto específico em que os projetos são desenvolvidos. Inspirados em guias amplamente utilizados, mas ajustados às particularidades de Machine Learning, padrões internos fornecem diretrizes claras para nomenclatura, organização, documentação e versionamento. Sua existência reduz ambiguidades, facilita a integração de novos(as) colaboradores(as) e reflete um compromisso com rigor técnico e metodológico.

Por fim, a análise crítica de código assume papel integrador ao incentivar a aplicação prática dos conceitos discutidos. Avaliar trechos de código, identificar problemas estruturais e conceituais e propor melhorias fundamentadas desenvolve no indivíduo leitor uma postura ativa e reflexiva. Essa prática contribui para a formação de profissionais capazes de não apenas escrever código funcional, mas de avaliar a qualidade, a clareza e a sustentabilidade de soluções de Machine Learning. A consolidação desses princípios fortalece a responsabilidade técnica e promove o desenvolvimento de projetos alinhados às exigências acadêmicas e profissionais contemporâneas.

REFERÊNCIAS

BURKOV, A. **Machine Learning Engineering**. [S. l.]: True Positive Inc., 2020.

COOKIECUTTER. **Cookiecutter data science**. 2026. Disponível em: <https://cookiecutter-data-science.drivendata.org/>. Acesso em: 25 mar. 2026.

GOOGLE. **Google Python Style Guide**. 2026. Disponível em: <https://chromium.googlesource.com/external/github.com/google/styleguide/+/f9347e1e9d79aee9cde0802fe178d72c8f87926c/pyguide.md>. Acesso em: 25 mar. 2026.

HUNT, A.; THOMAS, D. **The pragmatic programmer: your journey to mastery**. 2. ed. Boston: Addison-Wesley, 2019.

MARTIN, R. C. **Clean Code: a handbook of agile software craftsmanship**. Boston: Prentice Hall, 2008.

PYTHON SOFTWARE FOUNDATION. **PEP 257: docstring conventions**. [S. l.]: Python Software Foundation, 2001.

PYTHON SOFTWARE FOUNDATION. **PEP 8: style guide for Python code**. [S. l.]: Python Software Foundation, 2001.

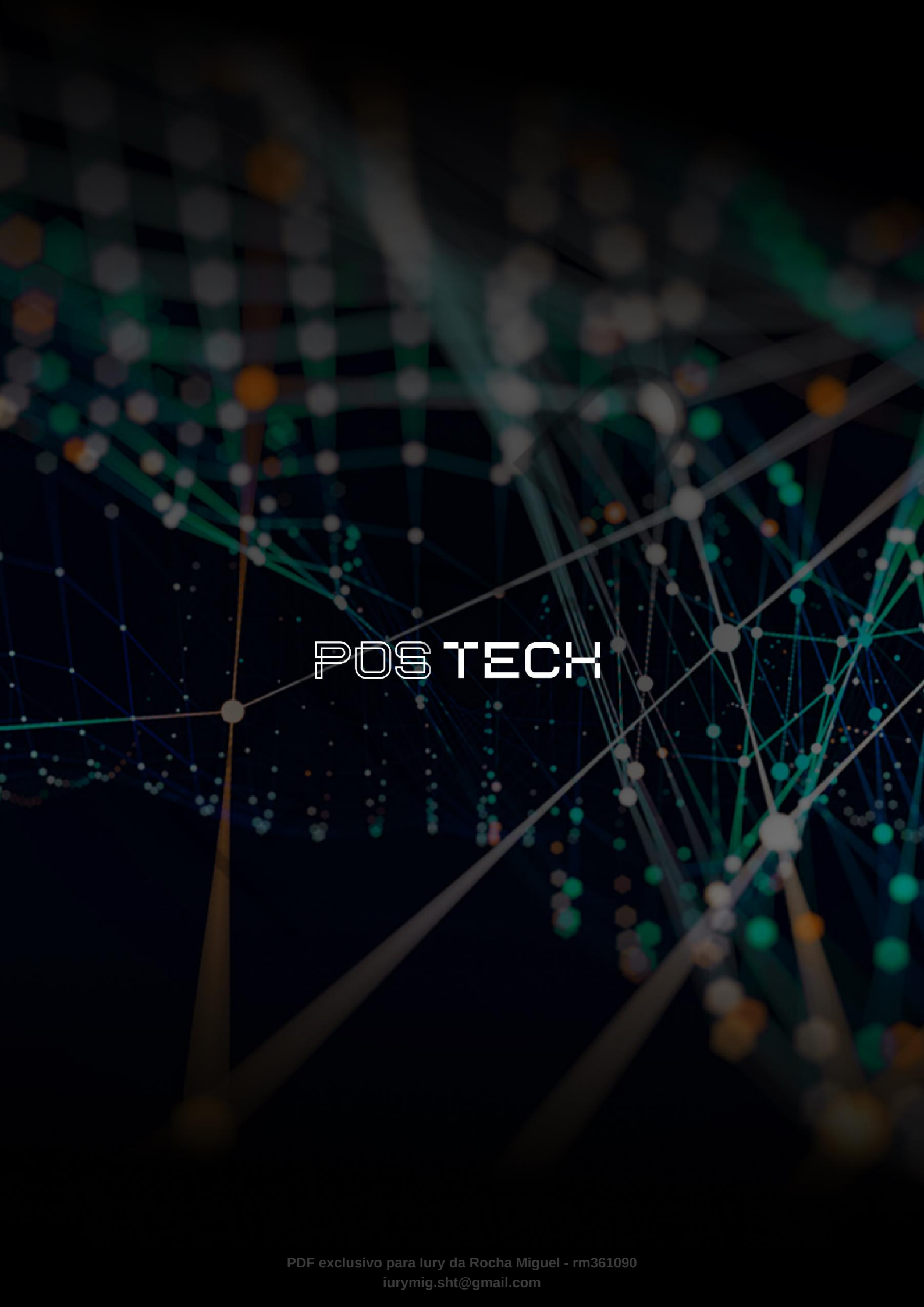
PYTHON SOFTWARE FOUNDATION. **Python documentation**. [S. l.]: Python Software Foundation, [s. d.].

RAJPUT, N. **How I Turned My Spaghetti Python Code Into a Maintainable Machine**. 2025. Disponível em: <https://medium.com/@hadiyolworld007/how-i-turned-my-spaghetti-python-code-into-a-maintainable-machine-e696cf4c9baa>. Acesso em: 25 mar. 2026.

PALAVRAS-CHAVE

Padronização. Reprodutibilidade. Sustentabilidade.

EMENDAS



POSTECH